

Forest Fire Detection — Complete Pipeline

Run all cells top to bottom. Each section builds on the previous one.

Section	What happens
Part 1	Install packages, set up paths, download + split dataset
Part 2	Train CNN baseline (EfficientNet-B4)
Part 3	Train Hybrid CNN+Transformer and ViT baseline
Part 4	Evaluate all 3 models, generate all report figures

```
# — STEP 0: Install packages (run once) —————
import subprocess, sys
packages = [
    'timm', 'albumentations', 'opencv-python', 'scikit-learn',
    'matplotlib', 'seaborn', 'pandas', 'tqdm', 'Pillow', 'kaggle', 'grad-cam'
]
subprocess.check_call([sys.executable, '-m', 'pip', 'install', '--quiet'] + packages)
print('All packages installed.')
```

All packages installed.

```
# — STEP 1: Imports, paths, device —————
import sys, os, random, shutil
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import torch
import torch.nn as nn
import timm
from sklearn.metrics import confusion_matrix, roc_curve, auc as sk_auc

try:
    import google.colab
    PROJECT_ROOT = Path('/content/forest_fire_detection')
    print('Environment : Google Colab')
except ImportError:
    PROJECT_ROOT = Path('.').resolve()
    print('Environment : Local')

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

RAW_DIR    = PROJECT_ROOT / 'data' / 'raw'
PROCESSED  = PROJECT_ROOT / 'data' / 'processed'
CKPT_BASE  = PROJECT_ROOT / 'outputs' / 'checkpoints'
RESULT_DIR = PROJECT_ROOT / 'outputs' / 'results'
PLOTS_DIR  = PROJECT_ROOT / 'outputs' / 'plots'

for d in [RAW_DIR, PROCESSED, CKPT_BASE, RESULT_DIR, PLOTS_DIR,
          CKPT_BASE/'baseline', CKPT_BASE/'hybrid', CKPT_BASE/'vit']:
    d.mkdir(parents=True, exist_ok=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device : {device}')
if device.type == 'cuda':
    print(f'GPU    : {torch.cuda.get_device_name(0)}')
else:
    print('WARNING: No GPU. Go to Runtime > Change runtime type > T4 GPU')

from src.dataset      import get_dataloaders, FireDataset
from src.model        import CNNBaseline
from src.hybrid_model import HybridCNNTransformer
from src.train         import train_model, compute_class_weights
from src.visualize     import plot_training_curves
from src.evaluate      import evaluate_model

print('PROJECT_ROOT :', PROJECT_ROOT)
print('Setup complete.')
```

Environment : Google Colab
Device : cuda
GPU : Tesla T4

```
PROJECT_ROOT : /content/forest_fire_detection
Setup complete.
```

▼ Part 1 — Data Preparation

```
# — PASTE YOUR KAGGLE CREDENTIALS HERE —————
KAGGLE_USERNAME = 'dhvanipatel160204' # ← from kaggle.json "username" field
KAGGLE_API_KEY = 'KGAT_52ee8a100280bc129b3ad35127e15e87' # ← from kaggle.json "key" field

# — FIX: organise images that were just extracted —————
import shutil
from pathlib import Path

IMAGE_EXTS = {'.jpg', '.jpeg', '.png', '.bmp'}
RAW_DIR = Path('/content/forest_fire_detection/data/raw')
fire_dir = RAW_DIR / 'fire'
no_fire_dir = RAW_DIR / 'no_fire'
fire_dir.mkdir(exist_ok=True)
no_fire_dir.mkdir(exist_ok=True)

def count_imgs(d):
    return sum(1 for f in Path(d).rglob('*') if f.suffix.lower() in IMAGE_EXTS)

# Show what was extracted (first 15 image paths)
print('Extracted image locations (sample):')
found = [f for f in RAW_DIR.rglob('*') if f.suffix.lower() in IMAGE_EXTS]
for f in found[:15]:
    print(' ', f.relative_to(RAW_DIR))
print(f'Total images found: {len(found)}')

# Copy using is_relative_to (fixes the startswith path prefix bug)
copied_fire = copied_nf = 0
for f in found:
    if f.is_relative_to(fire_dir) or f.is_relative_to(no_fire_dir):
        continue # already in destination
    parts = [p.lower() for p in f.parts]
    is_nf = any('non_fire' in p or 'no_fire' in p or 'non-fire' in p for p in parts)
    dst = no_fire_dir if is_nf else fire_dir
    shutil.copy2(f, dst / f.name)
    if is_nf: copied_nf += 1
    else: copied_fire += 1

print(f'\nCopied - fire: {copied_fire}, no_fire: {copied_nf}')
print(f'Final - fire: {count_imgs(fire_dir)}, no_fire: {count_imgs(no_fire_dir)}')
```

```
Extracted image locations (sample):
no_fire/non_fire.121.png
no_fire/non_fire.46.png
no_fire/non_fire.92.png
no_fire/non_fire.112.png
no_fire/non_fire.170.png
no_fire/non_fire.44.png
no_fire/non_fire.117.png
no_fire/non_fire.85.png
no_fire/non_fire.25.png
no_fire/non_fire.189.png
no_fire/non_fire.113.png
no_fire/non_fire.133.png
no_fire/non_fire.200.png
no_fire/non_fire.9.png
no_fire/non_fire.148.png
Total images found: 1998

Copied - fire: 755, no_fire: 244
Final - fire: 755, no_fire: 244
```

```
# — REPLACEMENT for Step 2 (paste this into a new cell) —————
import os, shutil, subprocess
from pathlib import Path

IMAGE_EXTS = {'.jpg', '.jpeg', '.png', '.bmp'}
RAW_DIR = Path('/content/forest_fire_detection/data/raw')
fire_dir = RAW_DIR / 'fire'
no_fire_dir = RAW_DIR / 'no_fire'
fire_dir.mkdir(parents=True, exist_ok=True)
no_fire_dir.mkdir(parents=True, exist_ok=True)

def count_imgs(d):
    return sum(1 for f in Path(d).rglob('*') if f.suffix.lower() in IMAGE_EXTS)

# — A: Upload kaggle.json if missing —————
```

```

kaggle_cfg = Path('/root/.kaggle/kaggle.json')
if not kaggle_cfg.exists():
    Path('/root/.kaggle/').mkdir(exist_ok=True)
    print('kaggle.json missing – a file picker will appear below.')
    print('Steps to get kaggle.json:')
    print('  1. kaggle.com → top-right avatar → Settings → API → Create New Token')
    print('  2. A kaggle.json file downloads to your computer')
    print('  3. Select it in the file picker that appears below')
    from google.colab import files
    uploaded = files.upload()
    for name, data in uploaded.items():
        kaggle_cfg.write_bytes(data)
    os.chmod(str(kaggle_cfg), 0o600)
    print('kaggle.json saved.')
else:
    print('kaggle.json already present.')

# — B: Download dataset —————
if count_imgs(fire_dir) < 100:
    print('Downloading fire-dataset ...')
    r = subprocess.run(
        ['kaggle', 'datasets', 'download', '-d', 'phylake1337/fire-dataset',
         '-p', str(RAW_DIR), '--unzip'],
        capture_output=True, text=True
    )
    print('STDOUT:', r.stdout[:400] if r.stdout else '(empty)')
    print('STDERR:', r.stderr[:400] if r.stderr else '(empty)')
    print('Exit code:', r.returncode)
else:
    print(f'Already have {count_imgs(fire_dir)} fire images, skipping download.')

# — C: Organise: copy images to fire/ and no_fire/ —————
for f in RAW_DIR.rglob('*'):
    if f.suffix.lower() not in IMAGE_EXTS: continue
    parts = [p.lower() for p in f.parts]
    is_nf = any('non_fire' in p or 'no_fire' in p or 'non-fire' in p for p in parts)
    in_dst = str(f).startswith(str(fire_dir)) or str(f).startswith(str(no_fire_dir))
    if not in_dst:
        dst = no_fire_dir if is_nf else fire_dir
        shutil.copy2(f, dst / f.name)

print(f'\nFinal – fire: {count_imgs(fire_dir)}, no_fire: {count_imgs(no_fire_dir)}')

```

kaggle.json missing – a file picker will appear below.
 Steps to get kaggle.json:
 1. kaggle.com → top-right avatar → Settings → API → Create New Token
 2. A kaggle.json file downloads to your computer
 3. Select it in the file picker that appears below

 kaggle.json

kaggle.json(application/json) - 78 bytes, last modified: 10/5/2026 - 100% done
 Saving kaggle.json to kaggle (1).json
 kaggle.json saved.
 Already have 755 fire images, skipping download.

Final – fire: 755, no_fire: 244

```

# — FIX: Inspect and reorganize the downloaded dataset —————
from pathlib import Path
import shutil

RAW_DIR = Path('/content/forest_fire_detection/data/raw')
fire_dir = RAW_DIR / 'fire'
no_fire_dir = RAW_DIR / 'no_fire'

IMAGE_EXTS = {'.jpg', '.jpeg', '.png', '.bmp'}

fire_dir.mkdir(parents=True, exist_ok=True)
no_fire_dir.mkdir(parents=True, exist_ok=True)

# Show the extracted structure first
print("Dataset structure:")
for p in list(RAW_DIR.rglob('*'))[:100]:
    print(" ", p)

# Organize based on folder/file names
fire_count = 0
no_fire_count = 0

for f in RAW_DIR.rglob('*'):
    if not f.is_file() or f.suffix.lower() not in IMAGE_EXTS:
        continue

```

```

# Don't re-process images already placed in destination folders
if fire_dir in f.parents or no_fire_dir in f.parents:
    continue

parts = [p.lower() for p in f.parts]

# Detect non-fire first
is_no_fire = any(
    key in part
    for part in parts
    for key in ['non_fire', 'no_fire', 'non-fire', 'nonfire']
)

# Detect fire
is_fire = any(
    key in part
    for part in parts
    for key in ['fire', 'fire_images', 'fire_images']
)

if is_no_fire:
    shutil.copy2(f, no_fire_dir / f.name)
    no_fire_count += 1
elif is_fire:
    shutil.copy2(f, fire_dir / f.name)
    fire_count += 1

print("\nCopied:")
print("Fire:", fire_count)
print("No-fire:", no_fire_count)

print("\nFinal counts:")
print("Fire:", sum(1 for f in fire_dir.iterdir()
                  if f.suffix.lower() in IMAGE_EXTS))
print("No-fire:", sum(1 for f in no_fire_dir.iterdir()
                      if f.suffix.lower() in IMAGE_EXTS))

```

Dataset structure:

```

/content/forest_fire_detection/data/raw/no_fire
/content/forest_fire_detection/data/raw/fire
/content/forest_fire_detection/data/raw/fire_dataset
/content/forest_fire_detection/data/raw/no_fire/non_fire.121.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.46.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.92.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.112.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.170.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.44.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.117.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.85.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.25.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.189.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.113.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.133.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.200.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.9.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.148.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.69.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.199.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.22.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.184.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.227.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.146.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.49.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.193.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.55.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.34.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.224.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.229.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.58.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.177.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.84.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.75.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.162.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.208.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.62.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.96.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.141.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.221.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.91.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.241.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.36.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.232.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.2.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.235.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.67.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.38.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.222.png

```

```

/content/forest_fire_detection/data/raw/no_fire/non_fire.73.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.60.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.98.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.12.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.173.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.47.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.100.png
/content/forest_fire_detection/data/raw/no_fire/non_fire.140.png

```

```

# — STEP 3: Stratified 70 / 15 / 15 split —————
SEED = 42
RATIOS = (0.70, 0.15, 0.15)
random.seed(SEED)
np.random.seed(SEED)

# Clear processed dir so re-runs are clean
if PROCESSED.exists():
    shutil.rmtree(str(PROCESSED))
PROCESSED.mkdir(parents=True)

CLASS_MAP = {'fire': fire_dir, 'no_fire': no_fire_dir}
split_counts = {s: {} for s in ('train', 'val', 'test')}

for cls, src in CLASS_MAP.items():
    images = sorted([p for p in src.rglob('*') if p.suffix.lower() in IMAGE_EXTS])
    random.shuffle(images)
    n = len(images)
    n1 = int(n * RATIOS[0])
    n2 = int(n * RATIOS[1])
    parts = {'train': images[:n1], 'val': images[n1:n1+n2], 'test': images[n1+n2:]}
    for split, files in parts.items():
        dest = PROCESSED / split / cls
        dest.mkdir(parents=True, exist_ok=True)
        for f in files:
            shutil.copy2(f, dest / f.name)
        split_counts[split][cls] = len(files)

print(f'{"Split":<8} {"fire":>8} {"no_fire":>10} {"Total":>8}')
print('-' * 42)
for split in ('train', 'val', 'test'):
    f = split_counts[split].get('fire', 0)
    nf = split_counts[split].get('no_fire', 0)
    print(f'{split:<8} {f:>8} {nf:>10} {f+nf:>8}')
print('Split complete.')

```

Split	fire	no_fire	Total
train	528	170	698
val	113	36	149
test	114	38	152

Split complete.

```

# — STEP 4: Sample images —————
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for row, cls in enumerate(['fire', 'no_fire']):
    imgs = sorted((PROCESSED / 'train' / cls).glob('*'))[:5]
    for col, p in enumerate(imgs):
        axes[row][col].imshow(Image.open(p).convert('RGB'))
        axes[row][col].set_title(cls, fontsize=9,
                                color='firebrick' if cls == 'fire' else 'steelblue')
        axes[row][col].axis('off')
plt.suptitle('Sample training images', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'sample_images.png', dpi=120, bbox_inches='tight')
plt.show()
print(f'Saved: {PLOTS_DIR}/sample_images.png')

```

Saved: /content/forest_fire_detection/outputs/plots/sample_images.png

```

# — STEP 5: Class balance chart —————
splits = ['train', 'val', 'test']
x, w = np.arange(len(splits)), 0.35
fc = [split_counts[s].get('fire', 0) for s in splits]
nfc = [split_counts[s].get('no_fire', 0) for s in splits]

fig, ax = plt.subplots(figsize=(8, 5))
b1 = ax.bar(x - w/2, fc, w, label='fire', color='firebrick', alpha=0.85)
b2 = ax.bar(x + w/2, nfc, w, label='no_fire', color='steelblue', alpha=0.85)
for bar in list(b1) + list(b2):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            str(int(bar.get_height())), ha='center', fontsize=9)
ax.set_xticks(x); ax.set_xticklabels(splits)

```

```
ax.set_ylabel('Images'); ax.set_title('Class balance per split')
ax.legend(); ax.grid(axis='y', alpha=0.4)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'class_balance.png', dpi=120, bbox_inches='tight')
plt.show()
tf, tnf = fc[0], nfc[0]
if tf > 0 and tnf > 0:
    print(f'Train imbalance ratio: {max(tf,tnf)/min(tf,tnf):.2f}x (fire={tf}, no_fire={tnf})')
```

Train imbalance ratio: 3.11x (fire=528, no_fire=170)

```
# — STEP 6: Image resolution stats —————
all_imgs = list((PROCESSED / 'train').rglob('*.jpg')) + \
    list((PROCESSED / 'train').rglob('*.png'))
sample = random.sample(all_imgs, min(300, len(all_imgs)))
widths, heights = [], []
ch_sum = np.zeros(3); ch_sq = np.zeros(3); px = 0
for p in sample:
    arr = np.array(Image.open(p).convert('RGB'), dtype=np.float32) / 255.0
    h, w = arr.shape[:2]
    widths.append(w); heights.append(h)
    ch_sum += arr.reshape(-1, 3).sum(0)
    ch_sq += (arr ** 2).reshape(-1, 3).sum(0)
    px += h * w
mean = ch_sum / px
std = np.sqrt(ch_sq / px - mean ** 2)
print(f'Width — mean:{np.mean(widths):.0f} min:{np.min(widths)} max:{np.max(widths)}')
print(f'Height — mean:{np.mean(heights):.0f} min:{np.min(heights)} max:{np.max(heights)}')
print(f'Channel mean (R,G,B): {mean.round(4)}')
print(f'Channel std (R,G,B): {std.round(4)}')
print('\nPart 1 complete.')
```

Width — mean:1155 min:185 max:6720
 Height — mean:755 min:165 max:4608
 Channel mean (R,G,B): [0.4323 0.3248 0.2212]
 Channel std (R,G,B): [0.287 0.2501 0.2439]

Part 1 complete.

✓ Part 2 — CNN Baseline (EfficientNet-B4)

Phase 1: freeze backbone, train head 5 epochs. Phase 2: fine-tune all layers 20 epochs.

```
# — CNN: data loaders + model —————
cnn_loaders = get_data_loaders(data_dir=str(PROCESSED), batch_size=32, img_size=224, num_workers=0)
class_weights = compute_class_weights(cnn_loaders['train'].dataset)
print(f'Train: {len(cnn_loaders["train"].dataset)} Val: {len(cnn_loaders["val"].dataset)} Test: {len(cnn_loaders["test"].dataset)}')
print('Class weights:', class_weights)

cnn_model = CNNBaseline(backbone='efficientnet_b4', num_classes=2, pretrained=True)
total = sum(p.numel() for p in cnn_model.parameters())
trainable = sum(p.numel() for p in cnn_model.parameters() if p.requires_grad)
print(f'Total params: {total:,} | Trainable: {trainable:,}')
```

Train: 698 Val: 149 Test: 152
 Class weights: tensor([1.5129, 0.4871])
 /usr/local/lib/python3.13/dist-packages/albumentations/core/validation.py:114: UserWarning: ShiftScaleRotate is a deprecated transform. Use Affine or Affine2D instead.
 original_init(self, **validated_kwargs)
 /content/forest_fire_detection/src/dataset.py:25: UserWarning: Argument(s) 'fog_coef_lower, fog_coef_upper' are not supported by A.RandomFog(fog_coef_lower=0.1, fog_coef_upper=0.3, p=0.15),
 /content/forest_fire_detection/src/dataset.py:27: UserWarning: Argument(s) 'max_holes, max_height, max_width' are not supported by A.CoarseDropout(max_holes=4, max_height=32, max_width=32, p=0.2),
 Total params: 17,552,202 | Trainable: 17,552,202

```
# — CNN Phase 1: frozen backbone, 5 epochs —————
print('=== CNN PHASE 1: frozen backbone ===')
cnn_model.freeze_backbone()
cnn_model, cnn_h1 = train_model(
    model=cnn_model, data_loaders=cnn_loaders,
    num_epochs=5, lr=1e-3, class_weights=class_weights,
    save_dir=str(CKPT_BASE / 'baseline'), device=device,
)
print('Phase 1 done.')
```

=== CNN PHASE 1: frozen backbone ===
 Training on: cuda

Epoch 1/5

```

train loss=0.5748 acc=0.8110
val   loss=0.4272 acc=0.9463
*** Checkpoint saved (val_acc=0.9463) ***

```

Epoch 2/5

```

-----
train loss=0.4063 acc=0.9092
val   loss=0.3210 acc=0.9329

```

Epoch 3/5

```

-----
train loss=0.3512 acc=0.9196
val   loss=0.2809 acc=0.9463

```

Epoch 4/5

```

-----
train loss=0.3181 acc=0.9167
val   loss=0.2682 acc=0.9396

```

Epoch 5/5

```

-----
train loss=0.3179 acc=0.9182
val   loss=0.2681 acc=0.9463

```

Best Validation Accuracy: 0.9463
Phase 1 done.

```

# --- CNN Phase 2: full fine-tune, 20 epochs ---
print('=== CNN PHASE 2: full fine-tune ===')
cnn_model.unfreeze_backbone()
cnn_model, cnn_h2 = train_model(
    model=cnn_model, dataloaders=cnn_loaders,
    num_epochs=20, lr=1e-4, class_weights=class_weights,
    save_dir=str(CKPT_BASE / 'baseline'), device=device,
)
print('Phase 2 done.')

```

=== CNN PHASE 2: full fine-tune ===
Training on: cuda

Epoch 1/20

```

-----
train loss=0.4278 acc=0.9122
val   loss=0.3035 acc=0.9463
*** Checkpoint saved (val_acc=0.9463) ***

```

Epoch 2/20

```

-----
train loss=0.3019 acc=0.9286
val   loss=0.2168 acc=0.9463

```

Epoch 3/20

```

-----
train loss=0.2526 acc=0.9360
val   loss=0.1567 acc=0.9530
*** Checkpoint saved (val_acc=0.9530) ***

```

Epoch 4/20

```

-----
train loss=0.1919 acc=0.9464
val   loss=0.1222 acc=0.9664
*** Checkpoint saved (val_acc=0.9664) ***

```

Epoch 5/20

```

-----
train loss=0.1387 acc=0.9628
val   loss=0.1105 acc=0.9664

```

Epoch 6/20

```

-----
train loss=0.1373 acc=0.9658
val   loss=0.0901 acc=0.9732
*** Checkpoint saved (val_acc=0.9732) ***

```

Epoch 7/20

```

-----
train loss=0.1284 acc=0.9568
val   loss=0.0858 acc=0.9732

```

Epoch 8/20

```

-----
train loss=0.1169 acc=0.9568
val   loss=0.0751 acc=0.9732

```

Epoch 9/20

```

-----
train loss=0.0962 acc=0.9747
val   loss=0.0659 acc=0.9866

```

```
*** Checkpoint saved (val_acc=0.9866) ***
```

```
Epoch 10/20
```

```
-----
train loss=0.0972 acc=0.9732
val   loss=0.0663 acc=0.9866
```

```
# — CNN: save history + plot —————
cnn_history = {
    'train_loss': cnn_h1['train_loss'] + cnn_h2['train_loss'],
    'val_loss'   : cnn_h1['val_loss']   + cnn_h2['val_loss'],
    'train_acc'  : cnn_h1['train_acc']  + cnn_h2['train_acc'],
    'val_acc'    : cnn_h1['val_acc']    + cnn_h2['val_acc'],
}
torch.save(cnn_history, RESULT_DIR / 'baseline_history.pt')
plot_training_curves(
    history=cnn_history, title='CNN Baseline (EfficientNet-B4)',
    save_path=str(PLOTS_DIR / 'baseline_training_curves.png'),
)
best_cnn = max(cnn_history['val_acc'])
print(f'Best val accuracy: {best_cnn:.4f}')
print('CNN training complete.')
```

```
Saved: /content/forest_fire_detection/outputs/plots/baseline_training_curves.png
Best val accuracy: 0.9866
CNN training complete.
```

✓ Part 3 — Hybrid CNN+Transformer and ViT Baseline

```
# — Hybrid: data loaders + model —————
# batch_size=16 because hybrid model uses more GPU memory
hybrid_loaders = get_data loaders(data_dir=str(PROCESSED), batch_size=16, img_size=224, num_workers=0)
h_weights      = compute_class_weights(hybrid_loaders['train'].dataset)
```

```
hybrid = HybridCNNTransformer(
    num_classes=2, backbone='efficientnet_b4', pretrained=True,
    embed_dim=512, num_heads=8, num_layers=4, dropout=0.3,
)
total = sum(p.numel() for p in hybrid.parameters())
print(f'Hybrid model - {total:,} parameters')
```

```
Hybrid model - 31,179,594 parameters
/content/forest_fire_detection/src/hybrid_model.py:29: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False
  self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
```

```
# — Hybrid Phase 1: CNN frozen, 8 epochs —————
print('=== HYBRID PHASE 1: CNN frozen ===')
hybrid.freeze_backbone()
hybrid, h_hist1 = train_model(
    model=hybrid, data loaders=hybrid_loaders,
    num_epochs=8, lr=5e-4, class_weights=h_weights,
    save_dir=str(CKPT_BASE / 'hybrid'), device=device,
)
print('Hybrid Phase 1 done.')
```

```
=== HYBRID PHASE 1: CNN frozen ===
Training on: cuda
```

```
Epoch 1/8
```

```
-----
train loss=0.6727 acc=0.7587
val   loss=0.1886 acc=0.9329
*** Checkpoint saved (val_acc=0.9329) ***
```

```
Epoch 2/8
```

```
-----
train loss=0.3637 acc=0.8910
val   loss=0.3648 acc=0.8725
```

```
Epoch 3/8
```

```
-----
train loss=0.3437 acc=0.9099
val   loss=0.1640 acc=0.9463
*** Checkpoint saved (val_acc=0.9463) ***
```

```
Epoch 4/8
```

```
-----
train loss=0.3047 acc=0.9012
val   loss=0.1070 acc=0.9463
```

```
Epoch 5/8
```



```

-----
train  loss=0.2801  acc=0.9186
val    loss=0.2457  acc=0.9329

Epoch 6/8
-----
train  loss=0.2890  acc=0.9026
val    loss=0.0951  acc=0.9664
*** Checkpoint saved (val_acc=0.9664) ***

Epoch 7/8
-----
train  loss=0.1792  acc=0.9433
val    loss=0.1321  acc=0.9530

Epoch 8/8
-----
train  loss=0.2150  acc=0.9520
val    loss=0.1108  acc=0.9463

Best Validation Accuracy: 0.9664
Hybrid Phase 1 done.

```

```

# — Hybrid Phase 2: full fine-tune with differential LR, 20 epochs —————
print('=== HYBRID PHASE 2: full fine-tune (diff LR) ===')
hybrid.unfreeze_backbone()
hybrid, h_hist2 = train_model(
    model=hybrid, dataloaders=hybrid_loaders,
    num_epochs=20, lr=1e-4, backbone_lr_multiplier=0.1,
    class_weights=h_weights,
    save_dir=str(CKPT_BASE / 'hybrid'), device=device,
)
print('Hybrid Phase 2 done.')

```

```

=== HYBRID PHASE 2: full fine-tune (diff LR) ===
Training on: cuda

```

```

Epoch 1/20
-----
train  loss=0.2548  acc=0.9390
val    loss=0.1573  acc=0.9597
*** Checkpoint saved (val_acc=0.9597) ***

Epoch 2/20
-----
train  loss=0.1777  acc=0.9637
val    loss=0.0994  acc=0.9664
*** Checkpoint saved (val_acc=0.9664) ***

Epoch 3/20
-----
train  loss=0.2237  acc=0.9520
val    loss=0.0903  acc=0.9664

Epoch 4/20
-----
train  loss=0.2063  acc=0.9390
val    loss=0.0642  acc=0.9664

Epoch 5/20
-----
train  loss=0.1590  acc=0.9593
val    loss=0.0623  acc=0.9664

Epoch 6/20
-----
train  loss=0.1809  acc=0.9622
val    loss=0.0520  acc=0.9866
*** Checkpoint saved (val_acc=0.9866) ***

Epoch 7/20
-----
train  loss=0.1919  acc=0.9506
val    loss=0.0729  acc=0.9799

Epoch 8/20
-----
train  loss=0.1798  acc=0.9477
val    loss=0.1258  acc=0.9597

Epoch 9/20
-----
train  loss=0.2409  acc=0.9477
val    loss=0.0297  acc=0.9866

Epoch 10/20
-----
train  loss=0.1738  acc=0.9564

```

```
val    loss=0.0522  acc=0.9799
```

Epoch 11/20

```
# — Hybrid: save history + plot —————
hybrid_history = {
    'train_loss': h_hist1['train_loss'] + h_hist2['train_loss'],
    'val_loss'   : h_hist1['val_loss']   + h_hist2['val_loss'],
    'train_acc'  : h_hist1['train_acc']  + h_hist2['train_acc'],
    'val_acc'    : h_hist1['val_acc']    + h_hist2['val_acc'],
}
torch.save(hybrid_history, RESULT_DIR / 'hybrid_history.pt')
plot_training_curves(
    history=hybrid_history, title='Hybrid CNN+Transformer',
    save_path=str(LOTS_DIR / 'hybrid_training_curves.png'),
)
best_hybrid = max(hybrid_history['val_acc'])
print(f'Best val accuracy: {best_hybrid:.4f}')
print('Hybrid training complete.')
```

Saved: /content/forest_fire_detection/outputs/plots/hybrid_training_curves.png
Best val accuracy: 0.9933
Hybrid training complete.

```
# — ViT-B/16: build —————
vit = timm.create_model('vit_base_patch16_224', pretrained=True, num_classes=2)
print(f'ViT-B/16 params: {sum(p.numel() for p in vit.parameters()):,}')
```

model.safetensors: reconstructing file: 100% 346MB / 346MB, 28.9MB/s
model.safetensors: downloading bytes: 330MB, 27.3MB/s
ViT-B/16 params: 85,800,194

```
# — ViT Phase 1: head only, 5 epochs —————
print('=== VIT PHASE 1: head only ===')
for name, p in vit.named_parameters():
    p.requires_grad = 'head' in name
vit, v_hist1 = train_model(
    model=vit, dataloaders=hybrid_loaders,
    num_epochs=5, lr=1e-3, class_weights=h_weights,
    save_dir=str(CKPT_BASE / 'vit'), device=device,
)
print('ViT Phase 1 done.')
```

=== VIT PHASE 1: head only ===
Training on: cuda

Epoch 1/5

```
-----
train loss=0.1827 acc=0.9433
val   loss=0.0235 acc=0.9933
*** Checkpoint saved (val_acc=0.9933) ***
```

Epoch 2/5

```
-----
train loss=0.1039 acc=0.9855
val   loss=0.0307 acc=0.9866
```

Epoch 3/5

```
-----
train loss=0.0853 acc=0.9855
val   loss=0.0326 acc=0.9866
```

Epoch 4/5

```
-----
train loss=0.0597 acc=0.9884
val   loss=0.0440 acc=0.9799
```

Epoch 5/5

```
-----
train loss=0.0720 acc=0.9826
val   loss=0.0347 acc=0.9799
```

Best Validation Accuracy: 0.9933
ViT Phase 1 done.

```
# — ViT Phase 2: full fine-tune, 20 epochs —————
print('=== VIT PHASE 2: full fine-tune ===')
for p in vit.parameters():
    p.requires_grad = True
vit, v_hist2 = train_model(
    model=vit, dataloaders=hybrid_loaders,
    num_epochs=20, lr=1e-5, class_weights=h_weights,
```

```

        save_dir=str(CKPT_BASE / 'vit'), device=device,
    )
    print('ViT Phase 2 done.')

```

```

=== VIT PHASE 2: full fine-tune ===
Training on: cuda

```

Epoch 1/20

```

-----
train  loss=0.1364  acc=0.9767
val    loss=0.0021  acc=1.0000
*** Checkpoint saved (val_acc=1.0000) ***

```

Epoch 2/20

```

-----
train  loss=0.1324  acc=0.9869
val    loss=0.0009  acc=1.0000

```

Epoch 3/20

```

-----
train  loss=0.1028  acc=0.9826
val    loss=0.0458  acc=0.9799

```

Epoch 4/20

```

-----
train  loss=0.0385  acc=0.9898
val    loss=0.0088  acc=0.9933

```

Epoch 5/20

```

-----
train  loss=0.0308  acc=0.9971
val    loss=0.0073  acc=0.9933

```

Epoch 6/20

```

-----
train  loss=0.0105  acc=0.9971
val    loss=0.0002  acc=1.0000

```

Epoch 7/20

```

-----
train  loss=0.0193  acc=0.9971
val    loss=0.0296  acc=0.9933

```

Epoch 8/20

```

-----
train  loss=0.0227  acc=0.9985
val    loss=0.0026  acc=1.0000

```

Epoch 9/20

```

-----
train  loss=0.0256  acc=0.9971
val    loss=0.0169  acc=0.9933

```

Epoch 10/20

```

-----
train  loss=0.0026  acc=0.9985
val    loss=0.0022  acc=1.0000

```

Epoch 11/20

```

-----
train  loss=0.0004  acc=1.0000
val    loss=0.0315  acc=0.9933

```

```

# — ViT: save history + plot —————
vit_history = {
    'train_loss': v_hist1['train_loss'] + v_hist2['train_loss'],
    'val_loss'   : v_hist1['val_loss']   + v_hist2['val_loss'],
    'train_acc'  : v_hist1['train_acc']  + v_hist2['train_acc'],
    'val_acc'    : v_hist1['val_acc']    + v_hist2['val_acc'],
}
torch.save(vit_history, RESULT_DIR / 'vit_history.pt')
plot_training_curves(
    history=vit_history, title='ViT-B/16 Baseline',
    save_path=str(PLOTS_DIR / 'vit_training_curves.png'),
)
best_vit = max(vit_history['val_acc'])
print(f'Best val accuracy: {best_vit:.4f}')
print('ViT training complete.')

```

```

Saved: /content/forest_fire_detection/outputs/plots/vit_training_curves.png
Best val accuracy: 1.0000
ViT training complete.

```

✓ Part 4 — Evaluation and Visualisation

Loads the best checkpoint of all 3 models and generates every figure needed for the report.

```
# — Load best checkpoints —————
def load_ckpt(model, path, device):
    state = torch.load(path, map_location=device, weights_only=True)
    model.load_state_dict(state)
    return model.eval().to(device)

test_loaders = get_data loaders(data_dir=str(PROCESSED), batch_size=32, img_size=224, num_workers=0)
test_loader = test_loaders['test']
print(f'Test samples: {len(test_loader.dataset)}')

cnn_eval = load_ckpt(
    CNNBaseline(backbone='efficientnet_b4', num_classes=2, pretrained=False),
    CKPT_BASE / 'baseline' / 'best_model.pth', device
)
hybrid_eval = load_ckpt(
    HybridCNNTransformer(num_classes=2, pretrained=False),
    CKPT_BASE / 'hybrid' / 'best_model.pth', device
)
vit_eval = load_ckpt(
    timm.create_model('vit_base_patch16_224', pretrained=False, num_classes=2),
    CKPT_BASE / 'vit' / 'best_model.pth', device
)
print('All 3 models loaded.')
```

Test samples: 152
All 3 models loaded.

```
# — Evaluate all 3 models —————
print('Evaluating CNN baseline ...')
cnn_m = evaluate_model(cnn_eval, test_loader, device)
print('Evaluating Hybrid ...')
hybrid_m = evaluate_model(hybrid_eval, test_loader, device)
print('Evaluating ViT ...')
vit_m = evaluate_model(vit_eval, test_loader, device)
print('Evaluation complete.')
```

Evaluating CNN baseline ...

```
=====
              precision    recall  f1-score   support

   No Fire         0.97         0.97         0.97         38
    Fire         0.99         0.99         0.99        114

 accuracy              0.99         152
 macro avg           0.98         0.98         0.98         152
 weighted avg        0.99         0.99         0.99         152
```

AUC-ROC : 0.9984

Evaluating Hybrid ...

```
=====
              precision    recall  f1-score   support

   No Fire         0.97         0.95         0.96         38
    Fire         0.98         0.99         0.99        114

 accuracy              0.98         152
 macro avg           0.98         0.97         0.97         152
 weighted avg        0.98         0.98         0.98         152
```

AUC-ROC : 0.9975

Evaluating ViT ...

```
=====
              precision    recall  f1-score   support

   No Fire         1.00         0.97         0.99         38
    Fire         0.99         1.00         1.00        114

 accuracy              0.99         152
 macro avg           1.00         0.99         0.99         152
 weighted avg        0.99         0.99         0.99         152
```

AUC-ROC : 1.0000

Evaluation complete.

```
# — Results comparison table —————
rows = []
```

```

for name, m in [('CNN (EfficientNet-B4)', cnn_m),
                ('ViT-B/16', vit_m),
                ('Hybrid CNN+Transformer (Ours)', hybrid_m)]:
    rows.append({
        'Model' : name,
        'Accuracy' : round(m['accuracy'], 4),
        'Precision' : round(m['precision'], 4),
        'Recall' : round(m['recall'], 4),
        'F1-Score' : round(m['f1'], 4),
        'AUC-ROC' : round(m['auc_roc'], 4),
    })
df = pd.DataFrame(rows)
print(df.to_string(index=False))
df.to_csv(RESULT_DIR / 'comparison_table.csv', index=False)
print(f'Saved: {RESULT_DIR}/comparison_table.csv')

```

	Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
	CNN (EfficientNet-B4)	0.9868	0.9912	0.9912	0.9912	0.9984
	ViT-B/16	0.9934	0.9913	1.0000	0.9956	1.0000
	Hybrid CNN+Transformer (Ours)	0.9803	0.9826	0.9912	0.9869	0.9975

Saved: /content/forest_fire_detection/outputs/results/comparison_table.csv

```

# — Confusion matrix (hybrid) —————
cm = confusion_matrix(hybrid_m['labels'], hybrid_m['preds'])
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Fire', 'Fire'], yticklabels=['No Fire', 'Fire'], ax=ax)
ax.set_title('Confusion Matrix - Hybrid Model')
ax.set_ylabel('True'); ax.set_xlabel('Predicted')
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'confusion_matrix_hybrid.png', dpi=150, bbox_inches='tight')
plt.show()
tn, fp, fn, tp = cm.ravel()
print(f'TP={tp} FP={fp} TN={tn} FN={fn}')
print(f'Recall (fire detected): {tp/(tp+fn):.4f}')

```

TP=113 FP=2 TN=36 FN=1
Recall (fire detected): 0.9912

```

# — ROC curves —————
fig, ax = plt.subplots(figsize=(7, 5))
colors = ['royalblue', 'seagreen', 'tomato']
for color, (name, m) in zip(colors, [
    ('CNN Baseline', cnn_m), ('ViT-B/16', vit_m), ('Hybrid (Ours)', hybrid_m)]):
    fpr, tpr, _ = roc_curve(m['labels'], m['probs'])
    roc_auc = sk_auc(fpr, tpr)
    ax.plot(fpr, tpr, color=color, lw=2, label=f'{name} (AUC={roc_auc:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax.set_xlabel='False Positive Rate', ylabel='True Positive Rate', title='ROC Curves - Test Set')
ax.legend(loc='lower right'); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'roc_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

```

# — Model comparison bar chart —————
model_names = ['CNN\nBaseline', 'ViT-B/16', 'Hybrid\n(Ours)']
metric_keys = ['accuracy', 'precision', 'recall', 'f1', 'auc_roc']
metric_lbls = ['Accuracy', 'Precision', 'Recall', 'F1', 'AUC-ROC']
all_m = [cnn_m, vit_m, hybrid_m]

x, w = np.arange(len(model_names)), 0.15
fig, ax = plt.subplots(figsize=(11, 5))
for i, (key, lbl) in enumerate(zip(metric_keys, metric_lbls)):
    vals = [m[key] for m in all_m]
    bars = ax.bar(x + i * w, vals, w, label=lbl)
    for b, v in zip(bars, vals):
        ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.003,
                f'{v:.3f}', ha='center', fontsize=7)
ax.set_xticks(x + w * 2); ax.set_xticklabels(model_names)
ax.set_ylim(0.5, 1.08); ax.set_ylabel('Score')
ax.set_title('Model Performance Comparison')
ax.legend(loc='lower right'); ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'model_comparison_bar.png', dpi=150, bbox_inches='tight')
plt.show()

```

```

# — Grad-CAM —————
try:
    from pytorch_grad_cam import GradCAM
    from pytorch_grad_cam.utils.image import show_cam_on_image
    from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

```

```

import torchvision.transforms as T

target_layer = [hybrid_eval.cnn.blocks[-1]]
cam = GradCAM(model=hybrid_eval, target_layers=target_layer)
tfm = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                 T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

fire_imgs = sorted((PROCESSED / 'test' / 'fire').glob('*'))[:4]
no_fire_imgs = sorted((PROCESSED / 'test' / 'no_fire').glob('*'))[:4]
samples = [(p, 'fire') for p in fire_imgs] + [(p, 'no_fire') for p in no_fire_imgs]

fig, axes = plt.subplots(2, len(samples), figsize=(len(samples) * 3, 6))
for col, (p, cls) in enumerate(samples):
    raw = np.array(Image.open(p).convert('RGB').resize((224, 224)))
    inp = tfm(Image.open(p).convert('RGB')).unsqueeze(0)
    tgt = [ClassifierOutputTarget(0 if cls == 'fire' else 1)]
    gs = cam(input_tensor=inp, targets=tgt)[0]
    ov = show_cam_on_image(raw / 255.0, gs, use_rgb=True)
    axes[0][col].imshow(raw); axes[0][col].set_title(cls, fontsize=9); axes[0][col].axis('off')
    axes[1][col].imshow(ov); axes[1][col].set_title('CAM', fontsize=9); axes[1][col].axis('off')
plt.suptitle('Grad-CAM - Hybrid Model', fontsize=12)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'gradcam.png', dpi=150, bbox_inches='tight')
plt.show()
print('Grad-CAM saved.')
except Exception as e:
    print(f'Grad-CAM skipped: {e}')

```

Grad-CAM saved.

```

# — Transformer attention maps —————
import torch.nn.functional as F
import torchvision.transforms as T
import matplotlib.cm as cm_module
import cv2

def get_attn_weights(model, inp, layer_idx=-1):
    """Hook-free attention extraction - replays the forward pass manually.

    F.multi_head_attention_forward expects seq-first inputs [S, B, D], so x_in
    is transposed before the call; the returned weights are already [B, H, S, S].
    """
    with torch.no_grad():
        feat = model.cnn(inp)
        feat = model.proj(feat)
        B, _, H, W = feat.shape
        tokens = feat.flatten(2).permute(0, 2, 1)
        tokens = tokens + model.pos_embed[:, :H * W, :]

        enc = model.transformer
        x = torch.cat([enc.cls_token.expand(B, -1, -1), tokens], dim=1) # [B, N+1, D]

        layers = list(enc.encoder.layers)
        n = len(layers)
        target = n + layer_idx if layer_idx < 0 else layer_idx

        for i in range(target):
            x = layers[i](x)

        layer = layers[target]
        mha = layer.self_attn
        x_in = layer.norm1(x) if layer.norm_first else x
        x_in = x_in.transpose(0, 1) # [B, N+1, D] → [N+1, B, D] (F expects seq-first)

        _, weights = F.multi_head_attention_forward(
            x_in, x_in, x_in,
            mha.embed_dim, mha.num_heads,
            mha.in_proj_weight, mha.in_proj_bias,
            mha.bias_k, mha.bias_v, mha.add_zero_attn,
            mha.dropout, mha.out_proj.weight, mha.out_proj.bias,
            training=False, need_weights=True, average_attn_weights=False,
        )
        return weights # [B, num_heads, N+1, N+1]

tfm = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                 T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
fire_imgs = sorted((PROCESSED / 'test' / 'fire').glob('*'))[:3]

fig, axes = plt.subplots(len(fire_imgs), 3, figsize=(10, 3.5 * len(fire_imgs)))
axes = np.array(axes).reshape(len(fire_imgs), 3)

for row, p in enumerate(fire_imgs):

```

```

raw = np.array(Image.open(p).convert('RGB').resize((224, 224)))
inp = tfm(Image.open(p).convert('RGB')).unsqueeze(0).to(device)

attn      = get_attn_weights(hybrid_eval, inp, layer_idx=-1)
cls_attn  = attn[0, :, 0, 1:].mean(0).cpu().numpy() # [N]
sz        = int(round(len(cls_attn) ** 0.5))
attn_map  = cls_attn.reshape(sz, sz)
attn_map  = (attn_map - attn_map.min()) / (attn_map.max() - attn_map.min() + 1e-8)
attn_up   = cv2.resize(attn_map, (224, 224))
overlay   = raw / 255.0 * 0.55 + cm_module.hot(attn_up)[: , :, :3] * 0.45

axes[row][0].imshow(raw); axes[row][0].set_title('Original')
axes[row][1].imshow(attn_map, cmap='hot'); axes[row][1].set_title('Attention Map')
axes[row][2].imshow(overlay); axes[row][2].set_title('Overlay')
for ax in axes[row]:
    ax.axis('off')

plt.suptitle('Transformer Attention Maps (CLS token)', fontsize=12)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'attention_maps.png', dpi=150, bbox_inches='tight')
plt.show()
print('Attention maps saved.')

```

/tmp/ipykernel_2360/1177734438.py:49: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (<https://matplotlib.org/3.1.1/using-matplotlib.html>) are not released until the user calls `plt.close()`. This can lead to a significant increase in memory usage. Consider using `plt.close()` to manually clear the stack. Alternatively, use `plt.close('all')` to clear all figures.

fig, axes = plt.subplots(len(fire_imgs), 3, figsize=(10, 3.5 * len(fire_imgs)))
Attention maps saved.

```

# — Failure cases —————
# Remove any stale pre/forward hooks left by earlier cells that crashed mid-run
for _m in hybrid_eval.modules():
    _m._forward_pre_hooks.clear()
    _m._forward_hooks.clear()

import torchvision.transforms as T

tfm = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
IDX = {'fire': 0, 'no_fire': 1}
REV = {0: 'fire', 1: 'no_fire'}

all_test = [(p, IDX[cls]) for cls in ['fire', 'no_fire']
             for p in (PROCESSED / 'test' / cls).glob('*')]
failures = []
hybrid_eval.eval()
with torch.no_grad():
    for p, true in all_test:
        t = tfm(Image.open(p).convert('RGB')).unsqueeze(0).to(device)
        prob = torch.softmax(hybrid_eval(t), 1)[0].cpu().numpy()
        pred = int(prob.argmax())
        if pred != true:
            failures.append((p, true, pred, float(prob[pred])))
failures.sort(key=lambda x: x[3], reverse=True)
fp_cnt = sum(1 for _, t, p, _ in failures if t == 1 and p == 0)
fn_cnt = sum(1 for _, t, p, _ in failures if t == 0 and p == 1)
print(f'Total failures: {len(failures)} / {len(all_test)} (FP={fp_cnt}, FN={fn_cnt})')
N = min(12, len(failures))
cols = 4
rows_n = -( - N // cols)
fig, axes = plt.subplots(rows_n, cols, figsize=(14, 3.5 * rows_n))
axes = np.array(axes).flatten()
for i, (p, true, pred, conf) in enumerate(failures[:N]):
    axes[i].imshow(Image.open(p).convert('RGB').resize((224, 224)))
    axes[i].set_title(f'True:{REV[true]}\nPred:{REV[pred]} (conf:{conf:.0%})', color='crimson', fontsize=8)
    axes[i].axis('off')
for ax in axes[N:]: ax.axis('off')
plt.suptitle('Failure Cases - Hybrid Model', fontsize=11)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'failure_cases.png', dpi=150, bbox_inches='tight')
plt.show()

```

Total failures: 146 / 152 (FP=35, FN=111)

```

# — Training curves for all 3 models side by side —————
histories = {
    'CNN Baseline': torch.load(RESULT_DIR / 'baseline_history.pt', map_location='cpu', weights_only=True),
    'ViT-B/16': torch.load(RESULT_DIR / 'vit_history.pt', map_location='cpu', weights_only=True),
    'Hybrid (Ours)': torch.load(RESULT_DIR / 'hybrid_history.pt', map_location='cpu', weights_only=True),
}
fig, axes = plt.subplots(1, 3, figsize=(18, 4))
for ax, (name, h) in zip(axes, histories.items()):
    ax.plot(h['train_acc'], label='Train')
    ax.plot(h['val_acc'], label='Val')
    ax.set_title(name)

```

```
ax.plot(h['val_acc'], label=val, linestyle='--',
best = max(h['val_acc']))
ax.set_title(f'{name}\nBest val: {best:.3f}', fontsize=10)
ax.set_xlabel('Epoch'); ax.set_ylabel('Accuracy')
ax.legend(); ax.grid(alpha=0.3)
plt.suptitle('Training Accuracy Curves', fontsize=13)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'all_training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print('All outputs saved to outputs/')
```

All outputs saved to outputs/

Done!

Figure	File
Confusion matrix	outputs/plots/confusion_matrix_hybrid.png
ROC curves	outputs/plots/roc_curves.png
Bar chart	outputs/plots/model_comparison_bar.png
Grad-CAM	outputs/plots/gradcam.png
Attention maps	outputs/plots/attention_maps.png
Failure cases	outputs/plots/failure_cases.png
Training curves	outputs/plots/all_training_curves.png
Results CSV	outputs/results/comparison_table.csv

```
import shutil
shutil.make_archive('/content/outputs', 'zip', '/content/forest_fire_detection/outputs')
from google.colab import files
files.download('/content/outputs.zip')
```